

9.4 Sorting with a Priority Queue

One application of priority queues is sorting, where we are given a sequence of elements that can be compared according to a total order relation, and we want to rearrange them in increasing order (or at least in nondecreasing order if there are ties). The algorithm for sorting a sequence S with a priority queue P is quite simple and consists of the following two phases:

1. In the first phase, we insert the elements of S as keys into an initially empty priority queue P by means of a series of n insert operations, one for each element.
2. In the second phase, we extract the elements from P in nondecreasing order by means of a series of n removeMin operations, putting them back into S in that order.

A Java implementation of this algorithm is given in Code Fragment 9.11, assuming that the sequence is stored as a positional list. (Code for a different type of collection, such as an array or an array list, would be similar.)

The algorithm works correctly for any priority queue P , no matter how P is implemented. However, the running time of the algorithm is determined by the running times of operations insert and removeMin, which do depend on how P is implemented. Indeed, pqSort should be considered more a sorting “scheme” than a sorting “algorithm,” because it does not specify how the priority queue P is implemented. The pqSort scheme is the paradigm of several popular sorting algorithms, including selection-sort, insertion-sort, and heap-sort, which we will discuss in this section.

```

1  /** Sorts sequence S, using initially empty priority queue P to produce the order. */
2  public static <E> void pqSort(PositionalList<E> S, PriorityQueue<E,?> P) {
3      int n = S.size();
4      for (int j=0; j < n; j++) {
5          E element = S.remove(S.first());
6          P.insert(element, null);      // element is key; null value
7      }
8      for (int j=0; j < n; j++) {
9          E element = P.removeMin().getKey();
10         S.addLast(element);          // the smallest key in P is next placed in S
11     }
12 }
```

Code Fragment 9.11: An implementation of a pqSort method that sorts elements of a positional list using an initially empty priority queue to produce the ordering.

9.4.1 Selection-Sort and Insertion-Sort

We next demonstrate how the pqSort scheme results in two classic sorting algorithms when using an unsorted or sorted list for a priority queue.

Selection-Sort

In Phase 1 of the pqSort scheme, we insert all elements into a priority queue P ; in Phase 2 we repeatedly remove the minimal element from P using the `removeMin` method. If we implement P with an unsorted list, then Phase 1 of pqSort takes $O(n)$ time, for we can insert each element in $O(1)$ time. In Phase 2, the running time of each `removeMin` operation is proportional to the size of P . Thus, the bottleneck computation is the repeated “selection” of the minimum element in Phase 2. For this reason, this algorithm is better known as *selection-sort*. (See Figure 9.7.)

	<i>Sequence S</i>	<i>Priority Queue P</i>
Input	(7, 4, 8, 2, 5, 3, 9)	()
Phase 1 (a)	(4, 8, 2, 5, 3, 9)	(7)
(b)	(8, 2, 5, 3, 9)	(7, 4)
⋮	⋮	⋮
(g)	()	(7, 4, 8, 2, 5, 3, 9)
Phase 2 (a)	(2)	(7, 4, 8, 5, 3, 9)
(b)	(2, 3)	(7, 4, 8, 5, 9)
(c)	(2, 3, 4)	(7, 8, 5, 9)
(d)	(2, 3, 4, 5)	(7, 8, 9)
(e)	(2, 3, 4, 5, 7)	(8, 9)
(f)	(2, 3, 4, 5, 7, 8)	(9)
(g)	(2, 3, 4, 5, 7, 8, 9)	()

Figure 9.7: Execution of selection-sort on sequence $S = (7, 4, 8, 2, 5, 3, 9)$.

As noted above, the bottleneck is in Phase 2 where we repeatedly remove an entry with smallest key from the priority queue P . The size of P starts at n and incrementally decreases with each `removeMin` until it becomes 0. Thus, the first `removeMin` operation takes time $O(n)$, the second one takes time $O(n-1)$, and so on, until the last (n^{th}) operation takes time $O(1)$. Therefore, the total time needed for the second phase is

$$O(n + (n-1) + \cdots + 2 + 1) = O\left(\sum_{i=1}^n i\right).$$

By Proposition 4.3, we have $\sum_{i=1}^n i = n(n+1)/2$. Thus, Phase 2 takes time $O(n^2)$, as does the entire selection-sort algorithm.

Insertion-Sort

If we implement the priority queue P using a sorted list, then the running time of Phase 2 improves to $O(n)$, for each operation `removeMin` on P now takes $O(1)$ time. Unfortunately, Phase 1 now becomes the bottleneck for the running time, since, in the worst case, each insert operation takes time proportional to the size of P . This sorting algorithm is therefore better known as **insertion-sort** (see Figure 9.8), for the bottleneck in this sorting algorithm involves the repeated “insertion” of a new element at the appropriate position in a sorted list.

	<i>Sequence S</i>	<i>Priority Queue P</i>
Input	(7, 4, 8, 2, 5, 3, 9)	()
Phase 1	(a) (4, 8, 2, 5, 3, 9)	(7)
	(b) (8, 2, 5, 3, 9)	(4, 7)
	(c) (2, 5, 3, 9)	(4, 7, 8)
	(d) (5, 3, 9)	(2, 4, 7, 8)
	(e) (3, 9)	(2, 4, 5, 7, 8)
	(f) (9)	(2, 3, 4, 5, 7, 8)
	(g) ()	(2, 3, 4, 5, 7, 8, 9)
Phase 2	(a) (2)	(3, 4, 5, 7, 8, 9)
	(b) (2, 3)	(4, 5, 7, 8, 9)
	⋮	⋮
	(g) (2, 3, 4, 5, 7, 8, 9)	()

Figure 9.8: Execution of insertion-sort on sequence $S = (7, 4, 8, 2, 5, 3, 9)$. In Phase 1, we repeatedly remove the first element of S and insert it into P . In Phase 2, we repeatedly perform the `removeMin` operation on P and add the returned element to the end of S .

Analyzing the running time of Phase 1 of insertion-sort, we note that it is

$$O(1 + 2 + \dots + (n-1) + n) = O\left(\sum_{i=1}^n i\right).$$

Again, by recalling Proposition 4.3, Phase 1 runs in $O(n^2)$ time, and hence, so does the entire insertion-sort algorithm.

Alternatively, we could change our definition of insertion-sort so that we insert elements starting from the end of the priority-queue list in Phase 1, in which case performing insertion-sort on a sequence that is already sorted would run in $O(n)$ time. Indeed, the running time of insertion-sort in this case is $O(n + I)$, where I is the number of **inversions** in the sequence, that is, the number of pairs of elements that start out in the input sequence in the wrong relative order.

9.4.2 Heap-Sort

As we have previously observed, realizing a priority queue with a heap has the advantage that all the methods in the priority queue ADT run in logarithmic time or better. Hence, this realization is suitable for applications where fast running times are sought for all the priority queue methods. Therefore, let us again consider the pqSort scheme, this time using a heap-based implementation of the priority queue.

During Phase 1, the i^{th} insert operation takes $O(\log i)$ time, since the heap has i entries after the operation is performed. Therefore, this phase takes $O(n \log n)$ time. (It could be improved to $O(n)$ with the bottom-up heap construction described in Section 9.3.4.)

During the second phase of method pqSort, the j^{th} removeMin operation runs in $O(\log(n - j + 1))$, since the heap has $n - j + 1$ entries at the time the operation is performed. Summing over all j , this phase takes $O(n \log n)$ time, so the entire priority-queue sorting algorithm runs in $O(n \log n)$ time when we use a heap to implement the priority queue. This sorting algorithm is better known as **heap-sort**, and its performance is summarized in the following proposition.

Proposition 9.4: *The heap-sort algorithm sorts a sequence S of n elements in $O(n \log n)$ time, assuming two elements of S can be compared in $O(1)$ time.*

Let us stress that the $O(n \log n)$ running time of heap-sort is considerably better than the $O(n^2)$ running time of selection-sort and insertion-sort.

Implementing Heap-Sort In-Place

If the sequence S to be sorted is implemented by means of an array-based sequence, such as an ArrayList in Java, we can speed up heap-sort and reduce its space requirement by a constant factor by using a portion of the array itself to store the heap, thus avoiding the use of an auxiliary heap data structure. This is accomplished by modifying the algorithm as follows:

1. We redefine the heap operations to be a *maximum-oriented* heap, with each position key being at least as *large* as its children. This can be done by re-coding the algorithm, or by providing a new comparator that reverses the outcome of each comparison. At any time during the execution of the algorithm, we use the left portion of S , up to a certain index $i - 1$, to store the entries of the heap, and the right portion of S , from index i to $n - 1$, to store the elements of the sequence. Thus, the first i elements of S (at indices $0, \dots, i - 1$) provide the array-list representation of the heap.
2. In the first phase of the algorithm, we start with an empty heap and move the boundary between the heap and the sequence from left to right, one step at a time. In step i , for $i = 1, \dots, n$, we expand the heap by adding the element at index $i - 1$.

3. In the second phase of the algorithm, we start with an empty sequence and move the boundary between the heap and the sequence from right to left, one step at a time. At step i , for $i = 1, \dots, n$, we remove a maximal element from the heap and store it at index $n - i$.

In general, we say that a sorting algorithm is *in-place* if it uses only a small amount of memory in addition to the sequence storing the objects to be sorted. The variation of heap-sort above qualifies as in-place; instead of transferring elements out of the sequence and then back in, we simply rearrange them. We illustrate the second phase of in-place heap-sort in Figure 9.9.

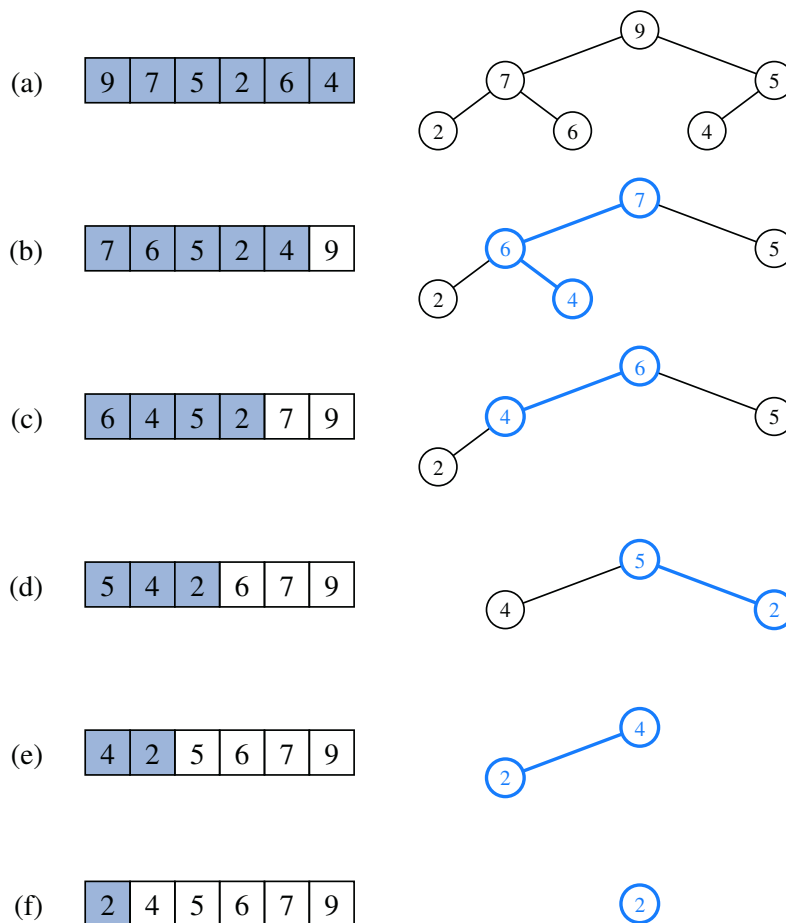


Figure 9.9: Phase 2 of an in-place heap-sort. The heap portion of each sequence representation is highlighted. The binary tree that each sequence (implicitly) represents is diagrammed with the most recent path of down-heap bubbling highlighted.